

Kean Flanagan
Linguistics 115, Spring 2026

Which Parts of Speech Are Frequently Code-Switched in Tagalog?

Introduction

Consider the following tweet: “Mag aaral talaga ako sa law subject ko na mamaya potangina gagoh pabagsak na akoh.” Law and subject, both words derived from English, are both inserted into a Tagalog syntactic frame without any morphological adaptation. This mixing of the languages is the result of code-switching, prevalent in Filipino Twitter but also prevalent in the daily lives of Tagalog-English bilingual speakers. It isn’t clear, however, what parts of speech is being replaced most frequently, or if it is even equal. Nouns, verbs, and adjectives could be interchangeable given the context, but how does it relate to Tagalog and English specifically?

I investigated that question using a corpus of tweets from late 2021 that carry language-proportion labels distinguishing Tagalog, English, and other-language content. The central research question is, Which parts of speech are most frequently code-switched in Tagalog-dominant Twitter discourse, and does the direction of switching, Tagalog-to-English versus English-to-Tagalog, affect which categories are transferred?

Background and Related Work

Code-switching between Tagalog and English, sometimes called Taglish, has a documented history in the Philippines that predates social media. Poplack’s (1980) constraint-based account of bilingual switching proposed that switches are more likely at syntactic boundaries where the two languages’ structures are congruent. Myers-Scotton (1993) proposed that one language serve as the matrix language, supplying the morphosyntactic frame, while the other acts as the embedded language, contributing content morphemes. Under this view, nouns and other open-class content words are the most borrowable category because they can be inserted into a matrix-language frame without violating its grammatical requirements.

More recent corpus work on Philippine social media has confirmed high rates of English nominal insertion. Bautista (2004) documented that borrowed English words in Filipino print media cluster heavily around nouns and adjectives. Social media introduces additional complexity: informal orthography, phonological respelling (e.g., “nxt” for “next” and “q” for “ako”), and emoji tokens challenge automatic part-of-speech tagging systems trained on formal text. Because spaCy’s English models are used here on what is largely Tagalog text, the POS labels must be treated as approximations rather than gold-standard annotations.

Methods

Data Collection

The source dataset is the tweet-ID file released alongside Herrera, Megal et al., a multilingual Twitter corpus with per-tweet language proportion annotations. Each tweet ID in the dataset is paired with percentages of language makeup (Tagalog proportion, English proportion, other proportion) derived from token-level language identification. The full file contains 21,150 tweet IDs.

Because the official Twitter/X API has become prohibitively expensive for undergraduate research (the free tier permits only one API call per fifteen minutes), tweet text was retrieved using the third-party service twitterapi.io, which provides access to public tweet data for \$0.15 per 1,000 requests. Three separate samples were drawn from the dataset based on the dominant language label: Tagalog-dominant tweets (Tagalog proportion $\geq 50\%$, English proportion $\geq 15\%$), English-dominant tweets (English proportion $\geq 50\%$), and other-dominant tweets (other proportion $\geq 50\%$). Target sample sizes were 1,000 tweets per group; however, tweet availability was substantially lower than anticipated, with only 99 Tagalog-dominant, 320 English-dominant, and 142 other-dominant tweets successfully retrieved, reflecting account deletions and suspensions accumulated since the dataset's 2021 creation. The resulting analysis treats these recovered tweets as an opportunity sample rather than a random sample.

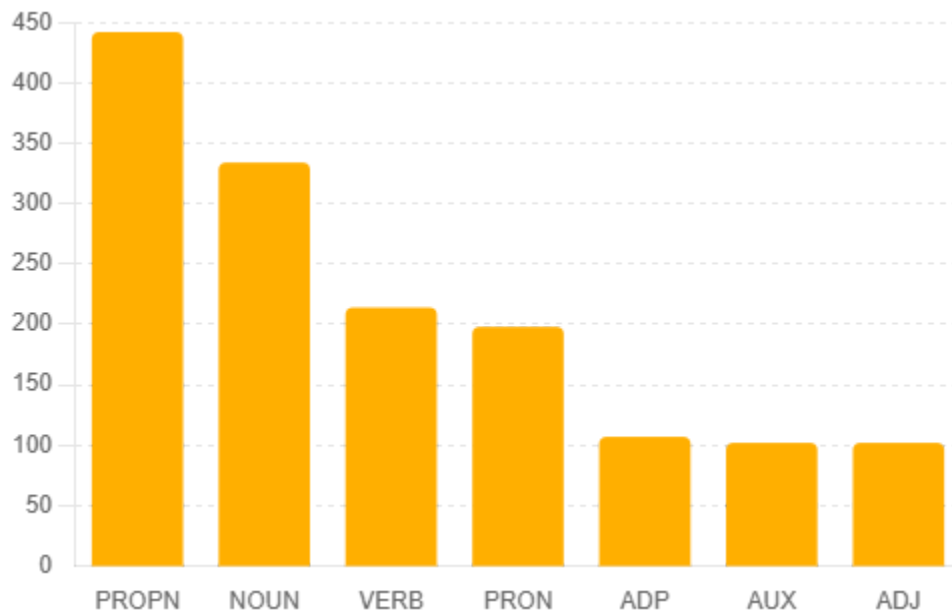
Annotation and Analysis

Recovered tweet text was processed with the Python library spaCy (version 3.x) using the `en_core_web_sm` model. Each token was assigned a Universal Dependencies part-of-speech tag (PROPN, NOUN, VERB, PRON, ADP, AUX, ADJ, DET, ADV, SCONJ, PART, CCONJ, INTJ, NUM, X). A token was classified as a candidate-switched token if its detected script language differed from the tweet's dominant language. Direction of switching was coded as 'tl' to 'en' (Tagalog matrix, English embedded token) or 'en' > 'tl' (English matrix, Tagalog embedded token). Tokens classified as "other," primarily emoji, numbers, and mentions, were excluded from the POS analysis.

Because spaCy's tagger was trained on English, Tagalog tokens are effectively out-of-vocabulary and receive English POS assignments by default. The vast majority of Tagalog tokens were therefore tagged PROPN, a known failure mode documented in prior work on low-resource multilingual NLP, which substantially inflates the PROPN count. This limitation is discussed further in the Discussion section.

Tagalog-Dominant Tweets (99 tweets, 1,852 candidate switched tokens)

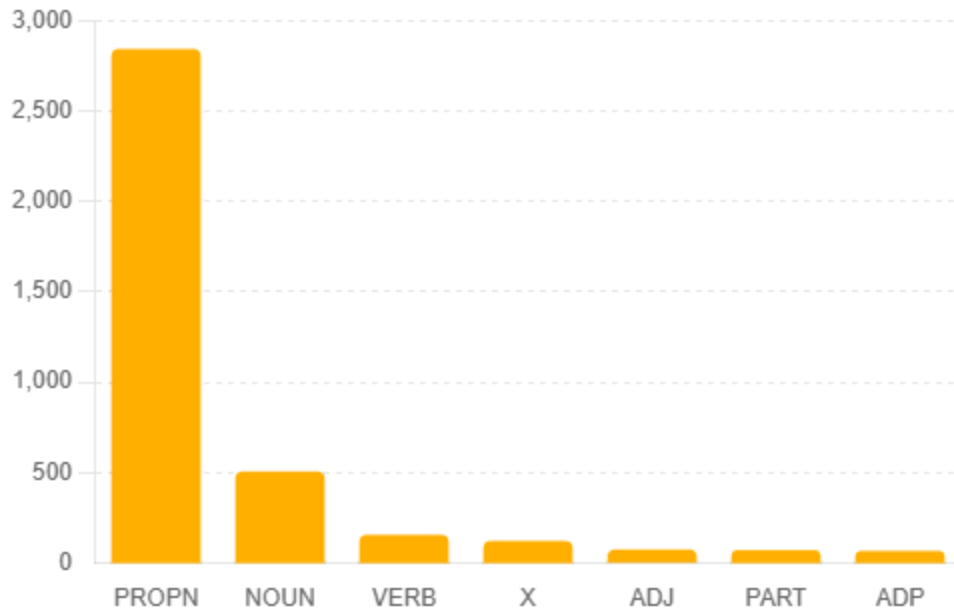
Among the 99 recovered Tagalog-dominant tweets, the most frequent POS labels assigned to candidate switched tokens were PROPN (442), NOUN (334), VERB (214), PRON (198), ADP (107), AUX (102), and ADJ (102). PROPN and NOUN together accounted for over 42% of all candidate-switched tokens. The direction of switching in these tweets was uniformly tl→en, consistent with Tagalog serving as the matrix language and English supplying lexical content. All 99 tweets were Tagalog-dominant by the classifier's labeling, with zero English-dominant tweets in this subsample.



[Figure 1: Bar chart of POS distribution among candidate-switched tokens, Tagalog-dominant tweets]

English-Dominant Tweets (320 tweets, 3,978 candidate switched tokens)

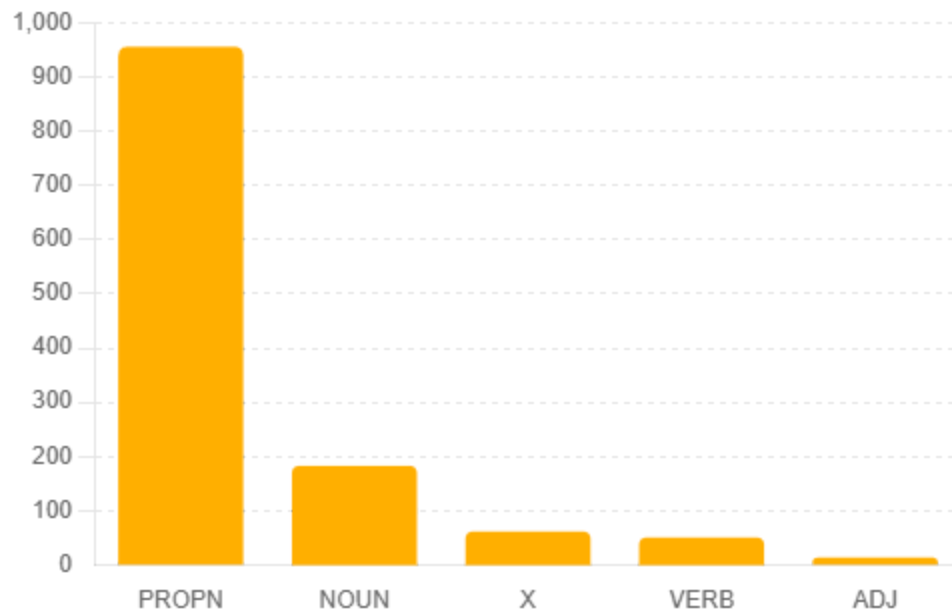
In the English-dominant sample, PROPN again led with 2,842 tokens, followed by NOUN (507), VERB (157), X (123), ADJ (76), PART (72), and ADP (68). English-dominant tweets showed a strongly skewed PROPN count relative to the other categories. The switching direction in these tweets was primarily 'en' > 'tl,' with 311 English-dominant tweets and only 9 Tagalog-dominant ones. The presence of Tagalog function words, particles such as “na,” “ng,” and “si,” and the discourse marker “di ba” within otherwise English tweets accounts for the ADP and PART counts.



[Figure 2: Bar chart of POS distribution among candidate-switched tokens, English-dominant tweets]

Other-Dominant Tweets (142 tweets, 1,320 candidate switched tokens)

The other-dominant sample was the smallest and the noisiest. PROP dominated at 955 tokens, with NOUN (183), X (62), VERB (51), and ADJ (14) trailing far behind. The high X count reflects non-standard and unrecognized tokens common in tweets that mix languages other than Tagalog or English: Indonesian, Spanish, Bahasa, and romanized Korean were all present in this subsample. The switching direction split 21 Tagalog-dominant from 121 English-dominant tweets, suggesting this group is better characterized as English-with-noise than as a third coherent language community.



[Figure 3: Bar chart of POS distribution among candidate-switched tokens, other-dominant tweets]

Discussion

Across all three subsamples, PROP and NOUN together constitute the plurality of candidate switched tokens, which is broadly consistent with the Myers-Scotton (1993) prediction that open-class content words, especially nouns, are the most permeable category in bilingual switching. English nouns appear in Tagalog-matrix tweets without phonological or morphological adaptation, a pattern that matches what Bautista (2004) observed in print media and what personal intuition confirms: English concepts like “law,” “mental health,” “album,” and “morning person” are inserted lexically because Tagalog either lacks a single-word equivalent or because the English term carries social prestige.

The near-absence of code-switched verbs deserves attention. In the Tagalog-dominant sample, VERB accounts for only 214 of 1,852 tokens (11.5%). Tagalog verb morphology is considerably more complex than English verb morphology: Tagalog verbs carry focus affixes (mag-, -um-, -in-, i-, -an) that signal the grammatical role of the noun phrase in focus, and tense is marked through reduplication and aspectual affixes rather than through distinct lexical forms. Inserting an English verb into a Tagalog clause would therefore require either stripping it of its Tagalog morphology, producing an ungrammatical sequence, or wrapping it in a light-verb construction (e.g., “nag-aacting,” “magdebut”). The data show some evidence of the latter strategy: tokens like “nag-aacting” and “magdebut” appear in the corpus with Tagalog prefixes attached to English stems. This morphological integration is itself a form of switching that the language identifier picks up as a Tagalog token, which may suppress the apparent VERB switching rate.

The other direction case—Tagalog words appearing in English-dominant tweets—is equally revealing. The most common switched tokens in English-matrix tweets are Tagalog discourse particles and pronouns: “di ba” (a tag question, literally “is it not”), “talaga” (an intensifier, roughly “really”), “parang” (a modal hedge, roughly “it seems like”), and “kayo/ako” (second- and first-person pronouns). These function words occupy closed grammatical classes that have no single-word English equivalents with the same pragmatic weight. “Di ba,” for instance, functions as a conversational appeal for agreement—closer to French “n’est-ce pas” than to English “right?”—and is retained in English-matrix tweets precisely because it is pragmatically untranslatable. This pattern points toward tag switching and intra-sentential switching as the dominant mechanisms in the ‘en’ > ‘tl’ direction, rather than the lexical insertion that characterizes ‘tl’ > ‘en’ switching.

The heavy PROPEN inflation is the principal methodological caveat. SpaCy’s English tagger assigns PROPEN to out-of-vocabulary tokens by default, and the majority of Tagalog tokens are OOV from the perspective of an English model. This means the PROPEN counts in all three subsamples are confounded: they include genuine proper nouns (names, brand names, hashtags, and mentions); Tagalog common nouns mis-tagged as PROPEN; and Tagalog verbs, particles, and pronouns also mis-tagged as PROPEN. A Tagalog-specific POS tagger or a cross-lingual model trained on code-switched Philippine data would substantially improve the precision of these counts.

Conclusion

This study used a recovered sample of 561 tweets from a 2021 multilingual Twitter corpus to examine which parts of speech are most frequently code-switched in Tagalog-English Twitter discourse. The results confirm that nominal categories, particularly nouns and items classified as proper nouns, dominate the switched-token inventory in Tagalog-matrix contexts, consistent with established accounts of borrowability in bilingual speech. The relative scarcity of switched verbs is attributable to Tagalog’s morphologically complex verb system, which makes bare-verb insertion grammatically difficult and instead produces hybrid forms with Tagalog affixes on English stems. In English-matrix contexts, Tagalog discourse particles and pronouns are the most common switched elements, suggesting a pragmatic motivation: these items express interactional meanings that lack direct English equivalents.

The analysis is limited by the low tweet-retrieval rate (roughly 10% of targeted samples), by reliance on an English-only POS tagger applied to code-switched text, and by the absence of manual annotation to validate the automatic language-identification labels. Future work should use a Tagalog-adapted or code-switching-specific NLP pipeline, expand the sample through higher-retrieval-rate data collection, and incorporate manual annotation. The annotated spreadsheets produced in this project provide a starting point for that effort.

References

Bautista, M. L. S. (2004). Tagalog-English code switching as a mode of discourse. *Asia Pacific Education Review*, 5(2), 226–233.

Herrera, L., Megal, Y., et al. (2021). Multilingual Twitter language-proportion dataset [Data set]. GitHub.

Myers-Scotton, C. (1993). *Duelling languages: Grammatical structure in codeswitching*. Clarendon Press.

Poplack, S. (1980). Sometimes I'll start a sentence in Spanish y termino en español: Toward a typology of code-switching. *Linguistics*, 18(7–8), 581–618.

twitterapi.io. (2025). Advanced Search API documentation. <https://docs.twitterapi.io>

Dataset

 codeswitched_tweets_tagalogmaj.xlsx

 codeswitched_tweets_othermaj.xlsx

 codeswitched_tweets_englishmaj.xlsx

Appendix

spacy_codeswitch_analysis.py

```

import re
from pathlib import Path
import pandas as pd
import spacy

INPUT_XLSX = Path('codeswitched_tweets.xlsx')
OUTPUT_DIR = Path('codeswitch_outputs')
DEDUP_BY_TWEET_ID = True

TAGALOG_FUNCTION_WORDS = {
    'ang','ng','sa','mga','si','ni','kay','kina','ito','iyan','iyon','dito','dyan','doon',
    'ako','ikaw','siya','kami','tayo','kayo','nila','namin','natin','mo','ko','ba','na',
    'pa','lang','din','rin','naman','kasi','pero','kahit','wala','meron','may','hindi','di',
    'oo','opo','eh','ano','sino','bakit','paano','kailan','saan','yung','nung','yan','yun'
}
TAGALOG_AFFIXES = (
    'mag','nag','pag','pinag','ipa','mapa','nak','um','in','-an','-han'
)

def simple_token_lang(token_text: str, dominant_lang: str, en_vocab: set[str]) -> str:
    t = token_text.lower()
    if not re.search(r"[A-Za-z]", t):
        return 'other'
    if t in TAGALOG_FUNCTION_WORDS:
        return 'tl'
    if t in en_vocab:
        return 'en'
    if any(t.startswith(pref) for pref in ['mag','nag','pag','ipa','mapa','pinag']):
        return 'tl'
    if dominant_lang == 'tl':
        return 'en' # unknown alphabetic token in Tagalog-dominant tweet is a likely insertion
    if dominant_lang == 'en':
        return 'tl' # unknown alphabetic token in English-dominant tweet is a likely insertion
    return 'unknown'

def main() -> None:
    OUTPUT_DIR.mkdir(exist_ok=True)

    df = pd.read_excel(INPUT_XLSX)
    df = df[df['Text'].notna()].copy()

```

```

if DEDUP_BY_TWEET_ID and 'Tweet ID' in df.columns:
    df = df.drop_duplicates(subset=['Tweet ID']).copy()

# Pick dominant matrix language using the spreadsheet percentages.
df['dominant_lang'] = df.apply(
    lambda r: 'tl' if float(r['Tagalog %']) >= float(r['English %']) else 'en', axis=1
)

nlp = spacy.load('en_core_web_sm', disable=['ner'])
en_vocab = {w.lower() for w in nlp.vocab.strings if re.fullmatch(r'[A-Za-z]+', w)}

records = []
for _, row in df.iterrows():
    doc = nlp(str(row['Text']))
    dominant_lang = row['dominant_lang']
    for tok in doc:
        if tok.is_space or tok.is_punct or tok.like_url or tok.like_email:
            continue
        token_lang = simple_token_lang(tok.text, dominant_lang, en_vocab)
        switched = token_lang in {'en', 'tl'} and token_lang != dominant_lang
        records.append({
            'Tweet ID': row.get('Tweet ID'),
            'Username': row.get('Username'),
            'dominant_lang': dominant_lang,
            'token': tok.text,
            'lemma': tok.lemma_,
            'pos': tok.pos_,
            'tag': tok.tag_,
            'token_lang': token_lang,
            'is_switched': switched,
            'text': row['Text'],
        })

token_df = pd.DataFrame(records)
switched_df = token_df[token_df['is_switched']].copy()

pos_summary = (
    switched_df.groupby(['dominant_lang', 'token_lang', 'pos'])
    .size()
    .reset_index(name='switched_token_count')
    .sort_values(['dominant_lang', 'token_lang', 'switched_token_count'], ascending=[True,
True, False])
)

overall_pos_summary = (

```

```

switched_df['pos'].value_counts()
.rename_axis('pos')
.reset_index(name='switched_token_count')
)

tweet_summary = pd.DataFrame({
    'metric': [
        'tweets_analyzed', 'candidate_switched_tokens', 'avg_tokens_per_tweet',
        'tagalog_dominant_tweets', 'english_dominant_tweets'
    ],
    'value': [
        len(df), len(switched_df), round(len(token_df) / max(len(df), 1), 2),
        int((df['dominant_lang'] == 'tl').sum()), int((df['dominant_lang'] == 'en').sum())
    ]
})

token_df.to_csv(OUTPUT_DIR / 'token_level_analysis.csv', index=False)
switched_df.to_csv(OUTPUT_DIR / 'candidate_switched_tokens.csv', index=False)
pos_summary.to_csv(OUTPUT_DIR / 'switched_pos_by_direction.csv', index=False)
overall_pos_summary.to_csv(OUTPUT_DIR / 'switched_pos_overall.csv', index=False)
tweet_summary.to_csv(OUTPUT_DIR / 'tweet_summary.csv', index=False)

print('\n=== Tweet summary ===')
print(tweet_summary.to_string(index=False))
print('\n=== Overall POS among candidate switched tokens ===')
print(overall_pos_summary.head(15).to_string(index=False))
print('\nWrote outputs to:', OUTPUT_DIR.resolve())

if __name__ == '__main__':
    main()

```

fetch_twitterapi_corpus.py

```

#!/usr/bin/env python3
import argparse
import ast
import sys
import time

import pandas as pd
import requests
from openpyxl import Workbook

```

```
from openpyxl.styles import Alignment, Font, PatternFill, Border, Side
from openpyxl.utils import get_column_letter
```

```
# — Config
```

```
SAMPLE_SIZE = 1000
RANDOM_SEED = 42
MIN_TAGALOG = 0.15
MIN_ENGLISH = 0.5
API_URL = "https://api.twitterapi.io/twitter/tweets"
BATCH_SIZE = 100
```

```
# — Colours
```

```
CLR_HEADER_BG = "1DA1F2"
CLR_HEADER_FG = "FFFFFF"
CLR_ROW_A = "E8F5E9"
CLR_ROW_B = "F5F5F5"
CLR_ERROR_BG = "FFEBEE"
CLR_ANNOT_BG = "FFDE7"
```

```
# — Helpers
```

```
def parse_split(raw: str):
    try:
        vals = ast.literal_eval(raw)
        return float(vals[0]), float(vals[1]), float(vals[2])
    except Exception:
        return None, None, None
```

```
def fetch_batch(ids: list[str], api_key: str) -> dict:
    headers = {"X-API-Key": api_key}
    params = {"tweet_ids": ",".join(ids)}

    try:
        resp = requests.get(API_URL, headers=headers, params=params, timeout=30)
    except requests.RequestException as e:
        print(f" [WARN] Request failed: {e}", flush=True)
    return {}
```

```

if resp.status_code == 401:
    sys.exit("[ERROR] Invalid API key.")

if resp.status_code == 429:
    print(" [Rate limited] Waiting 5s ...", flush=True)
    time.sleep(5)
    return fetch_batch(ids, api_key)

if not resp.ok:
    print(f" [WARN] HTTP {resp.status_code}: {resp.text[:200]}", flush=True)
    return {}

out = {}
for tweet in resp.json().get("tweets", []):
    tid = tweet.get("id")
    if tid:
        out[str(tid)] = tweet
return out

def thin_border():
    s = Side(style="thin", color="CCCCCC")
    return Border(left=s, right=s, top=s, bottom=s)

def header_font():
    return Font(bold=True, color=CLR_HEADER_FG, name="Arial", size=10)

def apply_header(ws, headers: list[str]):
    fill = PatternFill("solid", fgColor=CLR_HEADER_BG)
    for col, label in enumerate(headers, 1):
        cell = ws.cell(row=1, column=col, value=label)
        cell.font = header_font()
        cell.fill = fill
        cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
        cell.border = thin_border()
    ws.row_dimensions[1].height = 30

def main():
    parser = argparse.ArgumentParser(
        description="Fetch code-switched tweets via twitterapi.io and export to Excel."
    )
    parser.add_argument("--csv", required=True, help="Path to tweets_split_id.csv")

```

```

parser.add_argument("--api-key", required=True, help="Your twitterapi.io API key")
parser.add_argument("--output", default="codeswitched_tweets_othermaj.xlsx")
args = parser.parse_args()

# 1
print(f"[1/4] Loading {args.csv} ...")
df = pd.read_csv(args.csv, dtype={"id": str})
df[["tagalog", "english", "other"]] = df["split"].apply(
    lambda x: pd.Series(parse_split(x))
)
filtered = df[df["other"] >= 0.50]
sample = filtered.sample(SAMPLE_SIZE,
random_state=RANDOM_SEED).reset_index(drop=True)
print(f"    {len(filtered):,} rows match filter → sampling {SAMPLE_SIZE}")

# 2
print(f"\n[2/4] Fetching {SAMPLE_SIZE} tweets from twitterapi.io ...")
ids = sample["id"].tolist()
all_tweets = {}
n_batches = (len(ids) + BATCH_SIZE - 1) // BATCH_SIZE

for i in range(0, len(ids), BATCH_SIZE):
    batch = ids[i : i + BATCH_SIZE]
    batch_num = i // BATCH_SIZE + 1
    print(f"    Batch {batch_num}/{n_batches} ({len(batch)} IDs) ...", flush=True)
    result = fetch_batch(batch, args.api_key)
    all_tweets.update(result)
    time.sleep(0.2)

found = len(all_tweets)
missing = SAMPLE_SIZE - found
print(f"    Fetched: {found} | Unavailable/deleted: {missing}")

# 3
print("\n[3/4] Assembling data ...")
rows = []
for _, csv_row in sample.iterrows():
    tid = csv_row["id"]
    tweet = all_tweets.get(str(tid))
    error = None if tweet else "not returned by API"
    author = (tweet or {}).get("author", {})

    rows.append({
        "tweet_id": tid,
        "status": "unavailable" if error else "ok",
        "error_detail": error or "",
    })

```

```

"text":      (tweet or {}).get("text", ""),
"created_at": str((tweet or {}).get("createdAt", ""))[:10],
"api_lang":  (tweet or {}).get("lang", ""),
"author_id": author.get("id", ""),
"author_name": author.get("name", ""),
"username":  author.get("userName", ""),
"tagalog_pct": round(csv_row["tagalog"] * 100, 1),
"english_pct": round(csv_row["english"] * 100, 1),
"other_pct":  round(csv_row["other"] * 100, 1),
"likes":     (tweet or {}).get("likeCount", ""),
"retweets":  (tweet or {}).get("retweetCount", ""),
"replies":   (tweet or {}).get("replyCount", ""),
"quotes":    (tweet or {}).get("quoteCount", ""),
"views":     (tweet or {}).get("viewCount", ""),
"cs_words":  "",
"pos_tags":  "",
"switch_type": "",
"notes":     "",
})

```

```
# 4 (excel worksheet)
```

```
print(f"\n[4/4] Writing {args.output} ...")
```

```
wb = Workbook()
```

```
# sheet 1
```

```
ws1 = wb.active
```

```
ws1.title = "Tweets"
```

```
ws1.freeze_panes = "A2"
```

```
headers = [
```

```
    "Tweet ID", "Status", "Error Detail", "Text",
```

```
    "Date", "API Lang", "Author ID", "Author Name", "Username",
```

```
    "Tagalog %", "English %", "Other %",
```

```
    "Likes", "Retweets", "Replies", "Quotes", "Views",
```

```
    "Code-Switched Words ", "POS Tags ", "Switch Type ", "Notes ,
```

```
]
```

```
apply_header(ws1, headers)
```

```
col_widths = [20, 12, 28, 80, 12, 10, 20, 20, 18, 11, 11, 11, 8, 10, 8, 8, 10, 30, 20, 18, 25]
```

```
for i, w in enumerate(col_widths, 1):
```

```
    ws1.column_dimensions[get_column_letter(i)].width = w
```

```
fill_a = PatternFill("solid", fgColor=CLR_ROW_A)
```

```
fill_b = PatternFill("solid", fgColor=CLR_ROW_B)
```

```
fill_err = PatternFill("solid", fgColor=CLR_ERROR_BG)
```

```
fill_ann = PatternFill("solid", fgColor=CLR_ANNOT_BG)
```

```

for r_idx, row in enumerate(rows, 2):
    values = [
        row["tweet_id"], row["status"], row["error_detail"], row["text"],
        row["created_at"], row["api_lang"], row["author_id"],
        row["author_name"], row["username"],
        row["tagalog_pct"], row["english_pct"], row["other_pct"],
        row["likes"], row["retweets"], row["replies"], row["quotes"], row["views"],
        row["cs_words"], row["pos_tags"], row["switch_type"], row["notes"],
    ]
    is_error = row["status"] == "unavailable"
    row_fill = fill_err if is_error else (fill_a if r_idx % 2 == 0 else fill_b)

    for c_idx, val in enumerate(values, 1):
        cell = ws1.cell(row=r_idx, column=c_idx, value=val)
        cell.font = Font(name="Arial", size=9)
        cell.border = thin_border()
        cell.alignment = Alignment(vertical="top", wrap_text=(c_idx == 4))
        cell.fill = fill_ann if c_idx >= 18 else row_fill
        if c_idx in (10, 11, 12):
            cell.number_format = "0.0"

    ws1.row_dimensions[r_idx].height = 60 if not is_error else 20

ws1.auto_filter.ref = f"A1:{get_column_letter(len(headers))}1"

# sheet 2
ws2 = wb.create_sheet("Summary")
ws2.column_dimensions["A"].width = 32
ws2.column_dimensions["B"].width = 40

ok_rows = [r for r in rows if r["status"] == "ok"]
avg = lambda key: f"{sum(r[key] for r in ok_rows) / len(ok_rows):.1f}%" if ok_rows else
    "n/a"

summary_data = [
    ("Metric", "Value"),
    ("Source", "twitterapi.io"),
    ("Total sampled", SAMPLE_SIZE),
    ("Successfully fetched", found),
    ("Unavailable/deleted", missing),
    ("Filter: min Tagalog %", f"{int(MIN_TAGALOG*100)}%"),
    ("Filter: min English %", f"{int(MIN_ENGLISH*100)}%"),
    ("", ""),
    ("Avg Tagalog %", avg("tagalog_pct")),
    ("Avg English %", avg("english_pct")),

```

```

    ("Avg Other %",      avg("other_pct")),
    ("", ""),
    ("— Annotation Guide —", ""),
    ("Code-Switched Words", "Comma-separated words that switch language mid-tweet"),
    ("POS Tags",          "NOUN / VERB / ADJ / ADV per switched word, in same order"),
    ("Switch Type",       "inter-sentential / intra-sentential / tag-switching"),
    ("Notes",             "Free-form linguistic observations"),
]

hdr_fill = PatternFill("solid", fgColor=CLR_HEADER_BG)
for r_idx, (label, value) in enumerate(summary_data, 1):
    c1 = ws2.cell(row=r_idx, column=1, value=label)
    c2 = ws2.cell(row=r_idx, column=2, value=value)
    for c in (c1, c2):
        c.font = Font(name="Arial", size=10, bold=(r_idx == 1))
        c.alignment = Alignment(vertical="top", wrap_text=True)
        c.border = thin_border()
        if r_idx == 1:
            c.fill = hdr_fill
            c.font = header_font()
    ws2.row_dimensions[r_idx].height = 20

wb.save(args.output)
print(f"  Saved → {args.output}")
print(f"\n Done. Open the workbook and use the yellow annotation columns to tag
code-switched words.")

if __name__ == "__main__":
    main()

```